

# Spring Boot Filters - 2 (Quick Revision)

## 1. Modifying Response

- **Add response header:** `httpResponse.setHeader(name, value)` — easy, filter can add headers before sending response.
- **setHeader() vs addHeader():** `setHeader()` replaces old value; `addHeader()` adds another value without removing old one.
- **Why headers added before `chain.doFilter()`:** after `chain.doFilter()`, response may get **committed** (sent to client) — after that, header/status changes may not work. Check with `isCommitted()`.

## 2. Setting Response Status

- Filter can set status like `httpResponse.setStatus(SC_UNAUTHORIZED)`.
- Filter can also write full response body + `return;` (without calling chain) to **stop request** from reaching controller.
- `chain.doFilter()` = continue request. `return` without calling chain = stop request here.
- For real auth, use **Spring Security**, not custom filter.

## 3. Modifying Response Body

- Normal `HttpServletResponse` has **no** `getBody()` method — data is written directly to output stream/writer, not stored as object.
- To read/change body after controller runs, response must be **wrapped**.

## 4. ContentCachingResponseWrapper

- Spring class that **caches a copy** of response body so it can be read/modified after controller executes.
- Flow: wrap response → pass wrapper to chain → controller writes into wrapper → read cached bytes → modify → **must call** `copyBodyToResponse()` to send final data to real response (else client gets empty response).
- Steps: wrap → `chain.doFilter()` with wrapper → `getContentAsByteArray()` → `resetBuffer()` (clears body, keeps headers) → write new body → `copyBodyToResponse()`.

## 4.1 Limitations of manual body modification

- Body might be empty/not JSON/malformed, binary data (files/images) shouldn't be treated as text, large responses eat memory, headers like Content-Length may break, streaming responses don't fit caching.
- Better to use **Jackson** for JSON and **ResponseBodyAdvice** for global response changes instead of manual filter code.

## 5. Modifying Request Data

- Request headers/params are **read-only** — no `setHeader()` method on request.
- To change what downstream sees, use a **wrapper** instead of original request.

## 6. Request Wrapper Concept

- Custom class extends `HttpServletRequestWrapper`, overrides methods like `getHeader()`, `getParameter()`, `getInputStream()` etc.
- Downstream code receives wrapper, so it sees your modified values.
- Business logic transformations better done in service layer/validation, not filter.

## 7. Modifying Query Parameters

- No `setParameter()` method exists.
- Must override `getParameter()`, `getParameterMap()`, `getParameterValues()`, `getParameterNames()` **consistently** in wrapper (overriding only one may not work since Spring MVC may use another method).

## 8. Understanding Request Body

- No `getBody()` method. Body accessed as stream: `getInputStream()` or `getReader()`.
- Body is exposed as raw bytes because it can be JSON, XML, file, video etc. — server can't assume type/size in advance.
- Flow: client bytes → `ServletInputStream` → Spring picks `HttpMessageConverter` → Jackson (for JSON) → Java object (via `@RequestBody`).

## 9. Why Body Can't Be Read Repeatedly

- Stream has a **read position** — once read till end, it's empty, doesn't auto-reset.
- If filter reads body via `getReader()`/`getInputStream()`, controller may get **empty body** → error "Required request body is missing".

- Can't mix `getReader()` and `getInputStream()` on same request — both read same underlying stream.

## 10. Why No Auto-Rewind

- Auto-buffering every body (e.g. 2GB video) would waste memory, slow things down, cause DoS risk — so server doesn't do it automatically.

## 11. Re-reading/Replacing Request Body

- Need custom **request wrapper**: read original stream once → store bytes in byte array → wrapper returns fresh `ByteArrayInputStream` each time `getInputStream()/getReader()` is called → so body can be read multiple times.
- Use cases: logging, masking sensitive data, decrypting, sanitizing input.
- **Memory warning**: caching body = storing in memory; be careful with big uploads, file/multipart requests, passwords/tokens — never blindly log full request bodies.

## 12. Choosing Correct Layer

| Need                       | Better tool (not Filter)                                                                   |
|----------------------------|--------------------------------------------------------------------------------------------|
| Auth/authorization         | Spring Security                                                                            |
| Standard response wrapping | ResponseBodyAdvice                                                                         |
| Exception formatting       | @RestControllerAdvice                                                                      |
| JSON conversion            | Jackson config                                                                             |
| Controller arg creation    | HandlerMethodArgumentResolver                                                              |
| Validation                 | Bean Validation                                                                            |
| Business logic             | Service layer                                                                              |
| Low-level HTTP stuff       | Filter  |

## 13. Registering Filters — @Component

- Just annotate filter class with `@Component` → Spring Boot auto-registers it.
- Simple but applies **broadly** (usually `/`). Good for simple, broad cases.

## 14. Registering with FilterRegistrationBean

- Gives **explicit control**: URL patterns, order, name, dispatcher types, enable/disable.
- Steps: (1) create plain filter class (no `@Component`), (2) register manually in `@Configuration` class using `FilterRegistrationBean`, set filter, `addUrlPatterns()`, `setOrder()`, `setName()`.
- Can add **multiple URL patterns**.
- ⚠ Don't use both `@Component` AND manual registration for same filter — causes duplicate execution.

## 15. Controlling Filter Order

- `setOrder(1)` — **lower number runs first** on request.
- Request flow: Client → Filter(order1) → Filter(order2) → Controller.
- Response flow is **reverse** (like nested layers / onion) because each filter has code before & after `chain.doFilter()`.

## 16. Filtering Specific URLs Manually

- Inside `doFilter()`, check URI manually (`if (!uri.startsWith("/api/"))`) and skip logic if not needed.
- Filter still runs for every request but decides internally. Use when condition is simple or filter already broadly registered. For cleaner URL scoping, prefer `FilterRegistrationBean`.

## 17. OncePerRequestFilter

- Spring's abstract class made specifically for HTTP filters.
- Extend it, override `doFilterInternal(HttpServletRequest, HttpServletResponse, FilterChain)` — **no manual casting** needed (unlike plain `Filter`).
- Advantages: direct `HttpServletRequest/Response` access, prevents duplicate execution in same dispatch, Spring-specific hooks.
- Note: "Once" means once **per dispatch type** (REQUEST/ASYNC/ERROR/FORWARD/INCLUDE) — not literally only once ever, but for normal MVC requests it runs once as expected.

## 18. Excluding URLs with shouldNotFilter()

- Override `shouldNotFilter(request)` in `OncePerRequestFilter`.
- Return `true` → **skip** filter logic (`doFilterInternal` won't run).

- Return `false` → **run** filter logic.
- Example: `return !path.startsWith("/api/");` → only filters `/api/*` paths.

## 19. Comparing URL-Scoping Approaches

| Approach                                    | Best for                                                 |
|---------------------------------------------|----------------------------------------------------------|
| <code>FilterRegistrationBean</code>         | Clean broad URL-pattern control at registration          |
| Manual URI check in <code>doFilter()</code> | Small/custom runtime conditions                          |
| <code>shouldNotFilter()</code>              | Clean exclusion inside <code>OncePerRequestFilter</code> |

## 20. Filter vs OncePerRequestFilter

| Point                 | Filter                                              | OncePerRequestFilter                              |
|-----------------------|-----------------------------------------------------|---------------------------------------------------|
| API                   | Jakarta Servlet                                     | Spring                                            |
| Request/Response type | <code>ServletRequest/Response</code> (need casting) | <code>HttpServletRequest/Response</code> directly |
| Main method           | <code>doFilter()</code>                             | <code>doFilterInternal()</code>                   |
| Skip hook             | None built-in                                       | <code>shouldNotFilter()</code>                    |
| Dispatch protection   | Manual                                              | Built-in                                          |

→ In Spring Boot apps, **OncePerRequestFilter** is usually cleaner.

## 21. Practical Guidelines

**Use Filter for:** logging, trace/correlation IDs, CORS (if not using framework), compression, security chain integration, request timing, HTTP-boundary stuff.

**Avoid Filter for:** business logic, DB operations, DTO mapping, controller-specific logic, complex response restructuring (use `ResponseBodyAdvice`), auth (use Spring Security), blindly logging passwords/tokens/full bodies.

## 22. Final Mental Model (remember this!)

- **Headers:** response headers easy to add; request headers need wrapper.
- **Query params:** easy to read; need wrapper to modify.

- **Request body:** stream, read-once, needs caching+wrapper for re-read.
- **Response body:** written to output stream, needs wrapper to inspect/replace.
- **Filter registration:** `@Component` (simple/broad) | `FilterRegistrationBean` (explicit control) | `OncePerRequestFilter` (Spring-style base class).

## 23. Key One-Liners to Remember

1. No `chain.doFilter()` call = request stops there.
2. Add headers **before** response commits.
3. No `getBody()` on response — must wrap it (`ContentCachingResponseWrapper`).
4. Always call `copyBodyToResponse()` after modifying wrapped response body.
5. Request has no setters for headers/params — must use `HttpServletRequestWrapper`.
6. Reading request body in filter can **empty the stream** before controller reads it.
7. Re-readable body = cache bytes + override `getInputStream()`/`getReader()`.
8. Buffering bodies = memory + performance + privacy/security cost.
9. `FilterRegistrationBean` = URL pattern + order control.
10. Don't double-register same filter (`@Component` + manual).
11. `OncePerRequestFilter` = usually best choice in Spring Boot.
12. `shouldNotFilter()` = clean way to skip filter for certain paths.
13. Always use the **narrowest correct layer** — don't do everything in Filter.